



UNIVERSIDAD AUTÓNOMA DEL
ESTADO DE MÉXICO
FACULTAD DE INGENIERÍA



Licenciatura

Ingeniería en computación.

Unidad de aprendizaje:

Proyecto integral de comunicación de datos.

Grupo: 01.

Profesor:

Juan Carlos Escobar González.

Entregable 22: Check List UAT.

Plataforma inteligente de monitoreo de redes.

Equipo de trabajo:

Daniel Dávila Lara.

Jesús Omar Carranza Colín.

Fecha de entrega

14/mayo/2026

Proyecto/Entregable: Netguard. Plataforma inteligente de monitoreo de redes.

1. Criterios de aceptación – Nivel UAT.

- **CA-1 (Captura de tráfico):** El sistema debe capturar y registrar correctamente al menos el 90% de los paquetes enviados por los dispositivos ESP32 en un tiempo menor a 1 segundo.
- **CA-2 (Detección de anomalías):** El sistema debe identificar correctamente anomalías básicas de red, con una precisión mínima del 80%.
- **CA-3 (Visualización del dashboard):** El dashboard web debe mostrar las alertas generadas en tiempo real a partir de la detección de anomalías.

2. Checklist UAT

ID	Caso de prueba	Procedimiento	Resultado esperado	Estado (S/N)
01	Conectividad de dispositivos	Realizar un ping entre los ESP32 de la red.	La comunicación se realiza correctamente.	S
02	Captura de tráfico	Generar tráfico desde los ESP32 en la red.	Los paquetes se registran en el sistema.	S
03	Detección de dispositivos	Conectar un dispositivo cuya MAC no sea reconocida.	El dashboard muestra un mensaje de dispositivo no conocido.	S
04	Detección de anomalías	Ejecutar un ataque básico (por ejemplo, escaneo de puertos).	El sistema genera una alerta de anomalía.	S
05	Visualización en dashboard	Generar tráfico normal y anómalo.	El dashboard muestra el tráfico y las alertas.	S

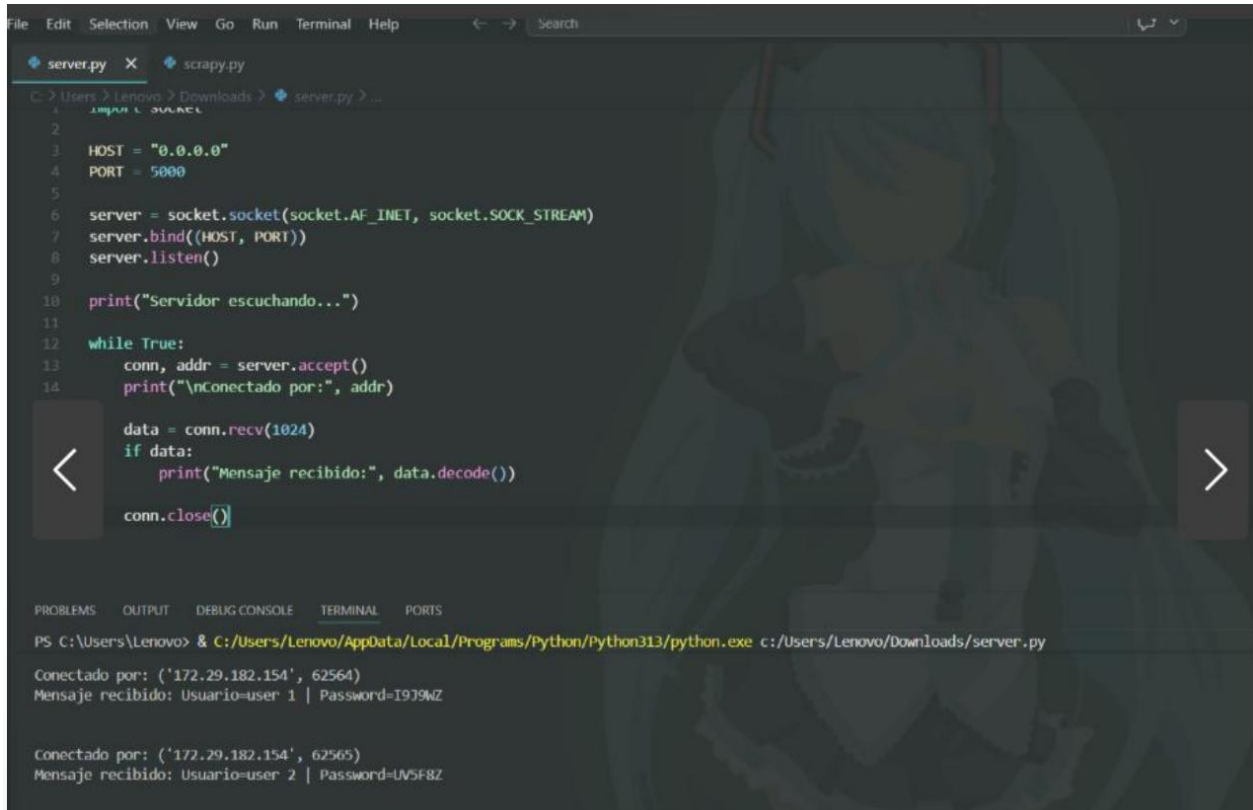
06	Autorización de dispositivo	Autorizar un nuevo dispositivo desde el dashboard.	El estado del dispositivo cambia a “ Autorizado ”.	S
07	Bloqueo de dispositivo	Bloquear dispositivos desde el dashboard.	El estado del dispositivo cambia a “ Bloqueado ”.	S
08	Funcionamiento general	Ejecutar una demostración completa del sistema.	El sistema opera correctamente de principio a fin.	S

3. Registro de Hallazgos.

ID	Hallazgo Identificado	Descripción Hallazgo	Impacto	Acción Correctiva	Responsable	Fecha Objetivo	Estado
01							
02							
03							
04							
05							

Anexos de pruebas.

1. Código para clientes y conexión de ESP32.



The screenshot shows a VS Code editor with a file named `server.py` open. The code is a simple Python server that listens on port 5000 and prints received data. The terminal shows the command to run the script and the output of the server.

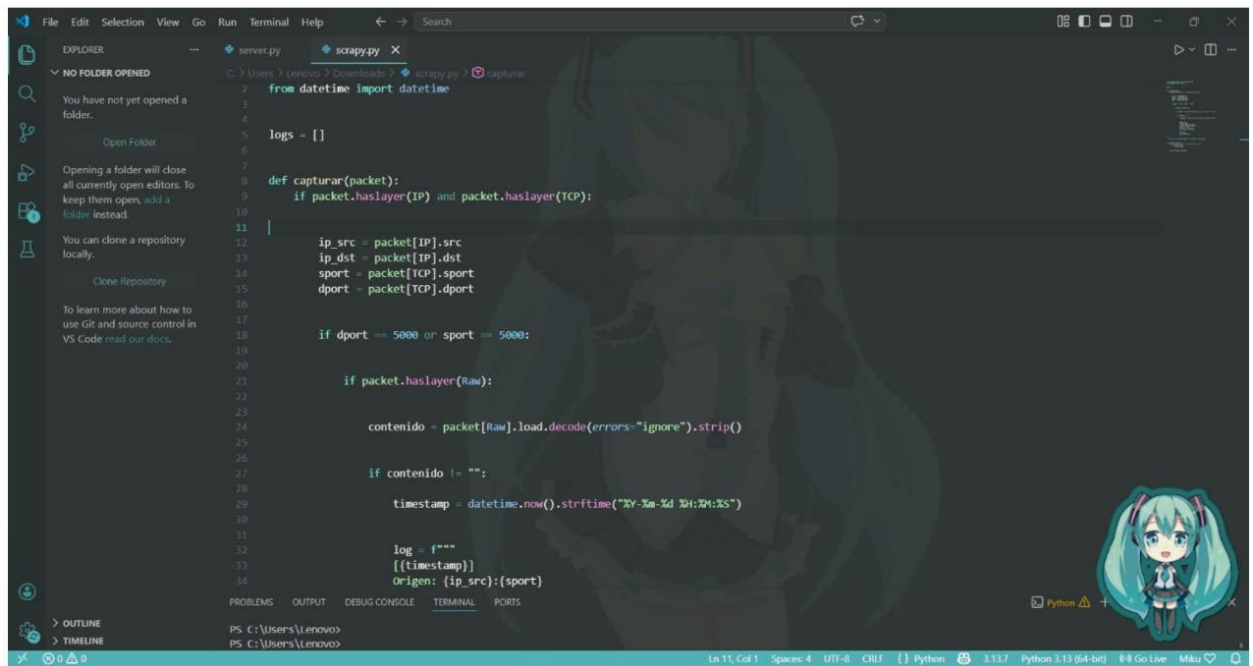
```
1 # Import the socket module
2
3 HOST = "0.0.0.0"
4 PORT = 5000
5
6 server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
7 server.bind((HOST, PORT))
8 server.listen()
9
10 print("Servidor escuchando...")
11
12 while True:
13     conn, addr = server.accept()
14     print("\nConectado por:", addr)
15
16     data = conn.recv(1024)
17     if data:
18         print("Mensaje recibido:", data.decode())
19
20     conn.close()
```

Terminal output:

```
PS C:\Users\Lenovo> & C:/Users/Lenovo/AppData/Local/Programs/Python/Python313/python.exe c:/Users/Lenovo/Downloads/server.py
Conectado por: ('172.29.182.154', 62564)
Mensaje recibido: Usuario=user 1 | Password=I9J9MZ

Conectado por: ('172.29.182.154', 62565)
Mensaje recibido: Usuario=user 2 | Password=UV5F8Z
```

2. Módulo de detección de anomalías



The screenshot shows a VS Code editor with a file named `scrapy.py` open. The code is a Python script that captures network packets and checks for anomalies. The terminal shows the command to run the script and the output of the script.

```
1 from datetime import datetime
2
3 logs = []
4
5 def capturar(packet):
6     if packet.haslayer(IP) and packet.haslayer(TCP):
7
8         ip_src = packet[IP].src
9         ip_dst = packet[IP].dst
10        sport = packet[TCP].sport
11        dport = packet[TCP].dport
12
13        if dport == 5000 or sport == 5000:
14
15            if packet.haslayer(Raw):
16
17                contenido = packet[Raw].load.decode(errors="ignore").strip()
18
19                if contenido != "":
20
21                    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
22
23                    log = f"""
24                    [(timestamp)]
25                    Origen: {ip_src}:{sport}
26                """
27
28                logs.append(log)
```

Terminal output:

```
PS C:\Users\Lenovo>
PS C:\Users\Lenovo>
```

3. Módulo de generación de tráfico.

```
traffic_generator.py X
C:\Users\jesus> Escritorio > traffic_generator.py > ...
1 from scapy.all import IP, TCP, UDP, ICMP, send
2 import random
3 import time
4
5 TARGET_IP = "192.168.1.100" # Cambia por la IP del servidor NetGuard
6
7
8 # -----
9 # 1. Tráfico normal TCP
10 # -----
11 def tcp_normal():
12     print("Enviando tráfico TCP normal...")
13     for _ in range(20):
14         packet = IP(dst=TARGET_IP) / TCP(
15             dport=80,
16             sport=random.randint(1024, 65535),
17             flags="S"
18         )
19         send(packet, verbose=False)
20         time.sleep(0.1)
21
22 # -----
23 # 2. Tráfico normal UDP
24 # -----
25 def udp_normal():
26     print("Enviando tráfico UDP normal...")
27     for _ in range(20):
28         packet = IP(dst=TARGET_IP) / UDP(
29             dport=53,
30             sport=random.randint(1024, 65535)
31         )
32         send(packet, verbose=False)
33         time.sleep(0.1)
34
35 # -----
36
37
```

4. Dashboard de análisis de red.

